Functional Programming

Kenjiro Taura

Contents

What is Functional Programming? 2
It is about Recursions! 5
Abstracting Computation Patterns by Functions 33
Inconvenient Truth: Deep Recursion, Stack Overflow, and Tail Recursion 40

What is Functional Programming?

1st attempt

> *… functional programming is a programming paradigm where programs are constructed by applying and composing functions.*
>
> — Wikipedia

- … well, I am almost always writing and applying functions, …

2nd attempt

- asked copilot: *describe the most important aspect of functional programming in less than 100 words*

  The most important aspect of functional programming is immutability. In functional programming, data is not changed after it is created. Instead, functions return new data structures, ensuring that the original data remains unchanged. This leads to more predictable and reliable code, as it eliminates side effects and makes debugging easier. Additionally, immutability facilitates parallel processing, as data consistency is maintained across threads.

It is about Recursions!

My take

- The most important aspect is a cetain “way of thinking” when you solve a problem,
- $\approx$ formulate the solution to the problem using **recursion**

An example

- Q: write a function that computes the sum of elements in an array
- A: a “procedural” Python version

```
def sum_array(a):
    n = len(a)
    s = 0
    for i in range(n):
        s = s + a[i]
    return s
```

The “thinking” behind the procedural version

- Well, to compute `a[0] + a[1] + ... + a[n-1]`,
  - start with `s = 0`, and

```
s = s + a[0]
s = s + a[1]
  ...
s = s + a[n-1]
```

- … now `s` should hold what we want
- Remember the time you were confused by the “equation”?

```
s = s + a[i]  # do you mean 0 = a[i] ??
```

A “functional” version

```
# a[i] + a[i+1] + ... + a[j-1]
def sum_range(a, i, j):
    if i == j:
        return 0
    else:
        return a[i] + sum_range(a, i + 1, j)

def sum_array(a):
    return sum_range(a, 0, len(a))
```

A (superficial) characteristics of the functional version

- No **updates** to variables (like `s = s + ...`)
- No **loops**

… but the point is not about **lack** of something

The “thinking” behind the functional version

- The key observation:

$$\text{sum of } a[0:n] = a[0] + (\text{sum of } a[1:n])$$

and you can compute (sum of $a[1:n]$) by a recursive call

- As a minor note, we defined a function to compute sum of an array **range** $a[i:j]$ by:

$$(\text{sum of } a[i:j]) = a[i] + (\text{sum of } a[i+1:j]),$$

- plus a trivial base case (i.e., $i = j \Rightarrow$ the sum is zero)

Reasoning correctness ≈ induction

```
# a[i] + a[i+1] + ... + a[j-1]
def sum_range(a, i, j):
    if i == j:
        return 0
    else:
        return a[i] + sum_range(a, i + 1, j)
```

- We like to establish `sum_range(`a, i, j`)` in fact returns

$$a[i] + \ldots + a[j-1] \quad (\star)$$

Reasoning correctness $\approx$ induction

1. $j - i = 0 \Rightarrow$ `sum_range(`a, i, j`)` returns `0` and $a[i] + \ldots + a[j-1] = 0$
2. Otherwise, assume the statement is true for $j - i < k$
   - Then, for $j - i = k$, `sum_range` returns

$$
\begin{aligned}
& a[i] + \text{sum_range}(a, i+1, j) \\
= \; & a[i] + (a[i+1] + \ldots + a[j-1]) && (\because \text{ induction hypothesis}) \\
= \; & a[i] + \ldots + a[j-1] && (\text{that is, } (\star))
\end{aligned}
$$

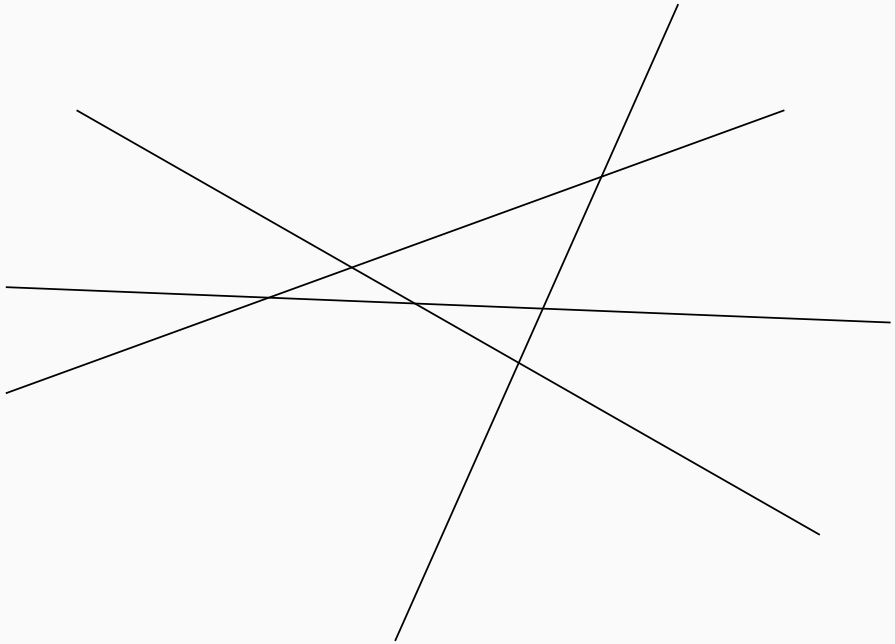
The “functional way” of problem solving

- $\approx$ solving a problem by recursive calls
- $\approx$ solving a problem, *assuming solutions to smaller cases are known*

very powerful for the same reason why solving math problems using **recurrence relation (漸化式)** and proving theorem by **induction (帰納法)** are very powerful

Solving problems with recurrence relation : an example

- Q: Draw n lines in a plane (no three lines intersect at a point). How many regions will result?



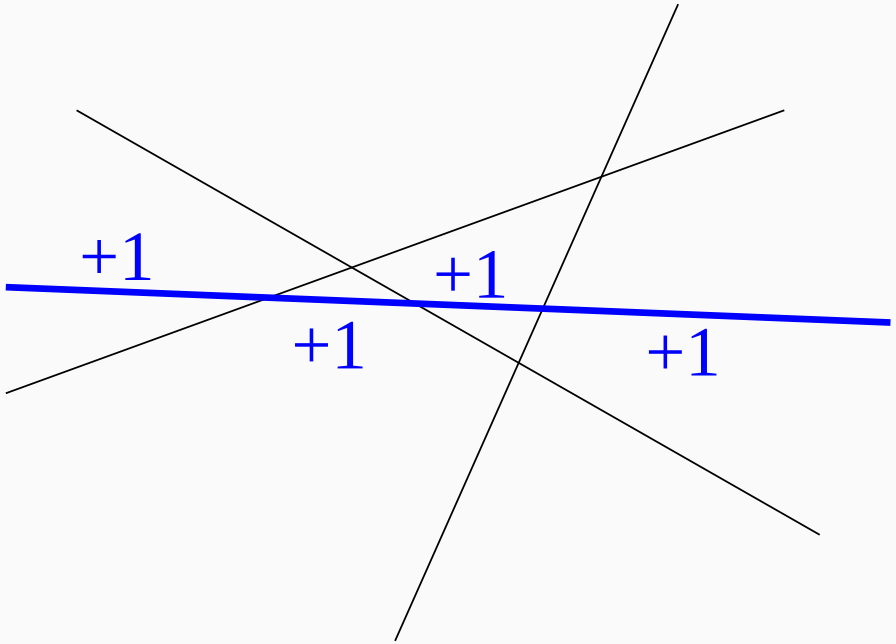
Solving problems with recurrence relation : an example

- Q: Draw n lines in a plane (no three lines intersect at a point). How many regions will result?

- A: Let the number of regions a_n. Then,

$$a_0 = 1,$$

$$a_n = a_{n-1} + n \quad (n > 0)$$



Into a code …

- Math:

$$a_0 = 1,$$
$$a_n = a_{n-1} + n \quad (n > 0)$$

- Code (can it be simpler?):

```
def n_regions(n):
    if n == 0:
        return 1
    else:
        return n_regions(n - 1) + n
```

Before you go, you might have already hate OCaml?

- In my past experiences (and this time), OCaml seems by far the most confusing language for you
- It *partly* stems from its being *functional* (i.e., encouraging functional programming and discouraging loops and variable updates), but it largely comes from syntax that has nothing to do with functional programming
- I address some of them before you make your mind

Reasons you might hate OCaml?

- function application by juxtaposition
- parentheses are still often necessary in function applications
- *partial applications* turn missing arguments on a function call into confusing type errors
- `f(a, b)` is not an immediate syntax error, but means something else
- different operator for integers and floating point numbers
- delimiters are often `;` not `,`
- delimiting with `,` means something else (tuple), not an immediate syntax error

Function application by juxtaposition

- `f x` instead of `f(x)`
- this alone does not make it that confusing

Parentheses are still often necessary in function applications

- `f x - 1` means $f(x) - 1$, not $f(x - 1)$
- so, you need to write `f (x - 1)` to pass $(x - 1)$ to f
- parentheses are necessary anyway here, but the purpose is different

Partial applications turn missing arguments on a function call into confusing type errors

- OCaml allows multi-parameter functions like

```
let f x y = x + y
```

to be applied to a single argument,e like

```
f 3
```

- it means *a function that takes y and returns $3 + y$*
- thus, if you somehow miss an argument to a function, you end up putting a function where you meant to put an ordinary value (like int)

Partial applications turn missing arguments on a function call into confusing type errors

- e.g., if you miss an argument to `g` below,

```
(g 1 2 3 ...) + 5
```

you get a *type error* saying “a value of function type where int is expected” or something like that

`f(a, b)` is not an immediate syntax error, but means something else

- `f(a, b)` is not a syntax error
- it applies `f` to `(a, b)` and it is a *tuple* consisting of `x` and `y`
- if `f` is defined as a two-parameter function like `let f x y = ...`, then you probably get a *type error* due to passing `(a, b)` to `x` (or the resulting expression missing the second argument `y`)

Different operator for integers and floating point numbers

- OCaml uses `+` for integers and `+.` for floating point numbers (same for `-`, `*`, `/`, etc.)
- it is partly to make it possible (or simple) to infer types of function parameters from function body
  - e.g., `let f x y = x + y` ⇒ `x` and `y` are `int` because `+` applies only for int
- you often write `x + y` when you in fact need to write `x +. y` etc.

Different operator for integers and floating point numbers

- in languages where the programmer has to declare types of function parameters (e.g., `x` and `y` above), this error is caught by “you apply `+` to floating point numbers”, but in OCaml, the error happens when you *apply* `f` (e.g., `f 3.1 4.1`) and the error message becomes “you put floating point numbers where integers are required”, probably not the error you committed)

Delimiters are often `;` not `,`

- you will later learn sequence data types (lists/arrays) and their syntax is not what you are accustomed to
  - Python: `[1, 2, 3]`
  - OCaml list: `[1; 2; 3]`
  - OCaml array: `[| 1; 2; 3 | ]`
- this alone is not that confusing

Delimiting with `,` means something else (tuple), not an immediate syntax erro

- in OCaml, `[1, 2, 3]` is not a syntax error
- it is a single-element list whose sole element is `1, 2, 3`, which is a tuple
- remember I talked about `f(x,y)`? `(x, y)` was a tuple
- thus, passing `[1, 2, 3]` to a function that expects a list of integers like `f [1,2,3]` is not a syntax error, but a type error saying "a value of type `(int * int * int) list` appears where `int list` is expected"

Divide-and-conquer

- A powerful problem-solving paradigm that
  1. *divides* the input into smaller subproblems,
  2. solves (*conquers*) each subproblem recursively, and
  3. combines their solutions to yield the solution

```
def solve(D):
    if D is trivially small:
        return trivial_solve(D)
    D0, D1, ... = divide(D)
    A0 = solve(D0); A1 = solve(D1); ...
    return combine(A0, A1, ...)
```

A textbook example (quicksort)

- Input : an array/list of n elements A
- Output : sort A (i.e., $A[0] \leq A[1] \leq \ldots \leq A[n-1]$)

```
def qs(A):
  if len(A) <= 1:
    return A
  piv = A[0]
  # divide
  lower  = [x for x in A[1:n] if x < piv]
  higher = [x for x in A[1:n] if x >= piv]
  # conquer & combine
  return qs(lower) + [piv] + qs(higher)
```

Other examples

- merge sort
- Discrete Fourier Transform (DFT)
  - $O(n^2)$ algorithm is trivial
  - FFT is a divide-and-conquer algorithm of $O(n \log n)$
- polynomial multiplication of two n-degree polynomials
  - $O(n^2)$ algorithm is trivial
  - Karatsuba algorithm is a divide-and-conquer algorithm of $O(n^{\log_2 3})$ algorithm?

Other examples

- matrix multiplication of two $n \times n$ matrices
  - $O(n^3)$ algorithm is trivial
  - Strassen algorithm is a divide-and-conquer algorithm of $\left(O\left(n^{\log_2 7}\right)\right)$

For Your Exercise …

- maximum segment sum
  - given an array A of n numbers, find p and q that maximizes sum of $A[p:q]$
  - $O(n^2)$ algorithm is trivial
  - can you come up with $O(n \log n)$ or $O(n)$ algorithm?
- inversion count
  - given an array A of n numbers, count the number of (i, j) pairs for which $A[i] > A[j]$
  - can you come up with $O(n \log n)$ algorithm?

Abstracting Computation Patterns by Functions

Common “Patterns”

e.g.,

```
def sum_square_pos(l):
  s = 0
  for x in l:
    if x > 0:
      s += x * x
  return s
```

- several common patterns in this code
  1. go over each element of an array (`for x in l`)
  2. do something when a condition is met (`if x > 0`)
  3. calculate on each element (`x * x`)
  4. reduce them into a single value (`s`)

Functional version (Python)

```python
def sum_square_pos(l):
  if l == []:
    return 0
  elif l[0] > 0:
    return l[0] * l[0] + sum_square_pos(l[1:])
  else:
    return sum_square_pos(l[1:])
```

Functional version (OCaml)

```
let rec sum_square_pos l = match l with
    [] -> 0
  | x :: r ->
    if x > 0
      x * x + sum_square_pos r
    else
      sum_square_pos r
```

- The same boilerplate for every different way of:
  - selecting elements (`x > 0`),
  - calculating a value for each selected element (`x * x`), and
  - reducing all values into one (`+`)

Higher-Order Functions on List (OCaml)

- `List.filter` $p\ l$ = list of elements x in l that satisfies $p\ x$
- `List.map` $f\ l$ = list of $f\ x$ for each x in l
- `List.fold_left` $r\ z\ l = r\ l_{n-1}\ (\ldots(r\ l_1\ (r\ l_0 z))))$
- `List.fold_right` $r\ z\ l = r\ l_0\ (\ldots(r\ l_{n-2}\ (r\ l_{n-1}\ z)))$
- With them and anonymous functions (`fun x -> ...`),

```
let rec sum_square_pos l =
  List.fold_left (fun x y -> x + y) 0
    (List.map (fun x -> x * x)
      (List.filter (fun x -> x > 0) l))
```

A Shorter Version

- OCaml supports:

1. "Function" versions of infix operators: `(+)`, `(<)`, ...
   - i.e., `(+) x y` ≡ `x + y`
   - ∴ `(+)` ≡ `fun x y -> x + y`
2. Partial applications. e.g.,
   - for `f x y = E`, two parameter function, `f x` ≡ `fun y -> E`
   - ∴ `(<) 0` ≡ `fun y -> (<) 0 y` (≡ `fun y -> 0 < y`)
3. Pipeline operator
   - `x |> f` ≡ `f x`

A Shorter Version

- Combined,

```
let sum_square_pos l =
  l |> List.filter ((<) 0)
    |> List.map (fun x -> x * x)
    |> List.fold_left (+) 0
```

- Note: the language you chose may or may not have similar functions builtin (you can roll it by yourself when it doesn't)

Inconvenient Truth: Deep Recursion, Stack Overflow, and Tail Recursion

Deep recursion may lead to stack overflow

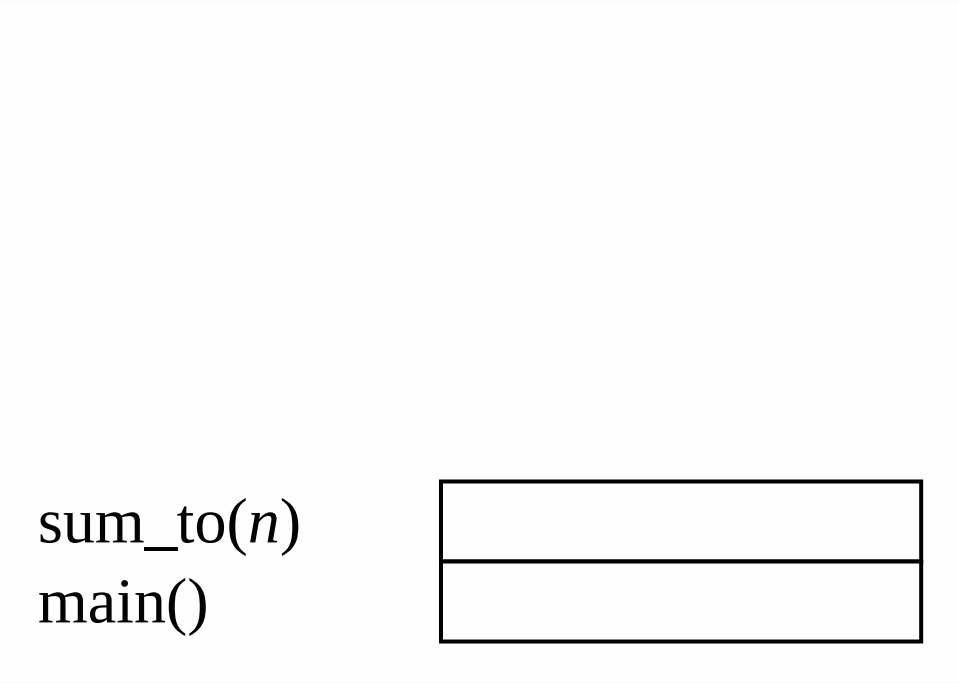
- e.g., to compute sum $1 + \ldots + n$

```
def sum_to(n):
  if n == 0:
    return 0
  else:
    return n + sum_to(n - 1)
```

```
>>> sum_to(1000)
    ... <snip> ...
    return n + sum_to(n - 1)
               ^^^^^^^^^^^^^^
  [Previous line repeated 996
more times]
RecursionError: maximum
recursion depth exceeded
```

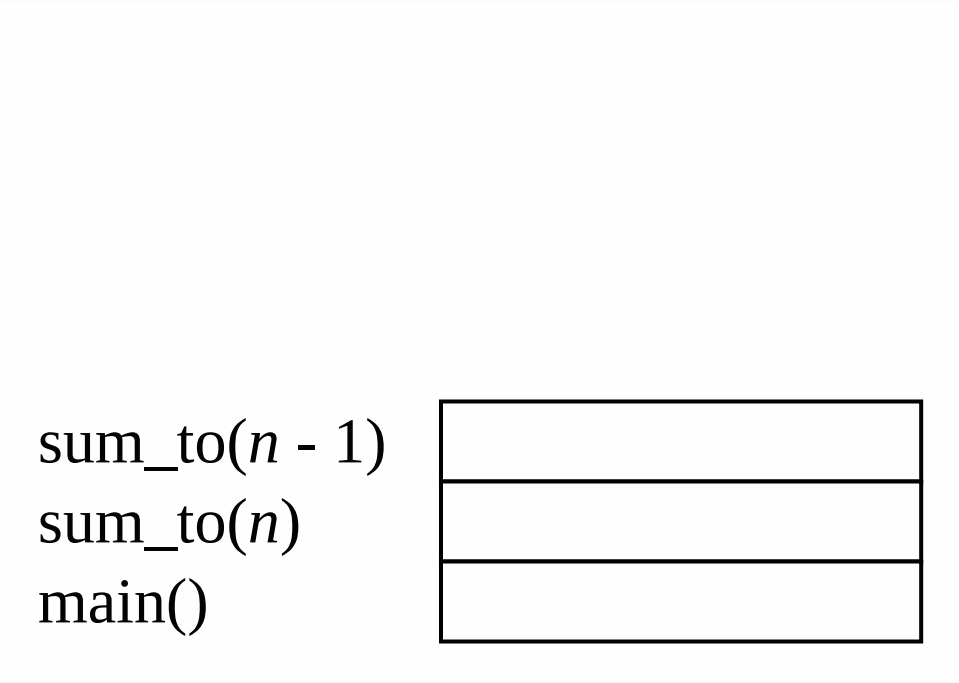
Why does it happen?

- A function call requires space for storing variables and intermediate values



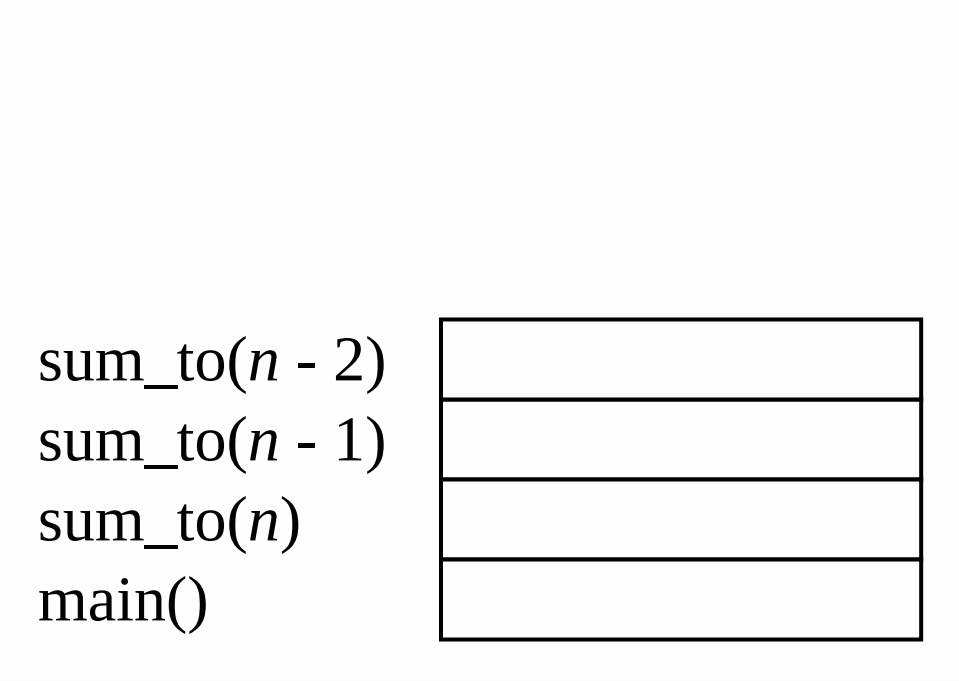
Why does it happen?

- A function call requires space for storing variables and intermediate values



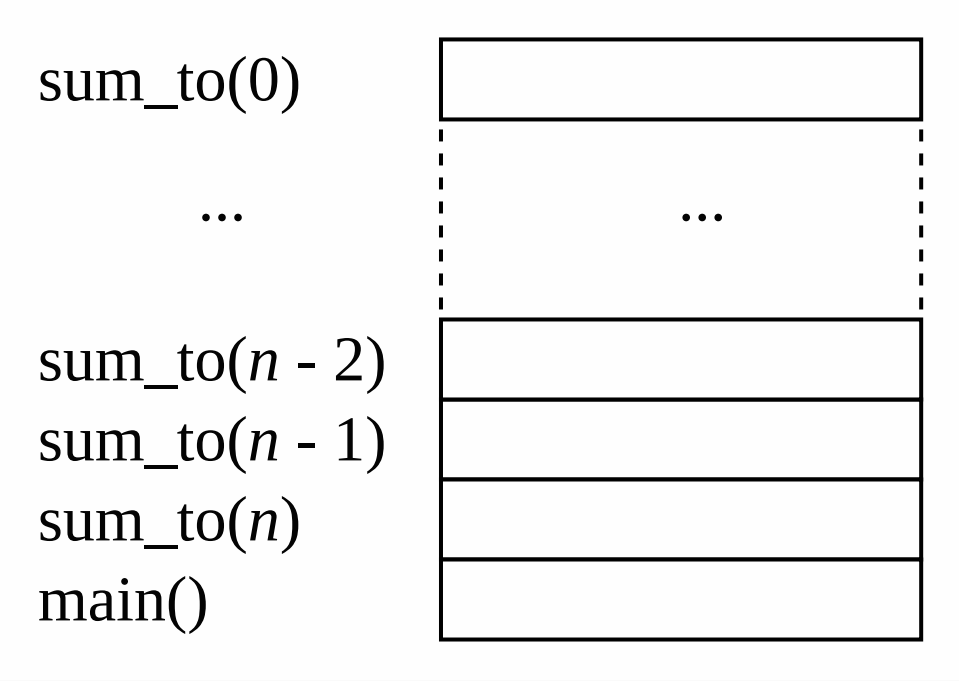
Why does it happen?

- A function call requires space for storing variables and intermediate values



Why does it happen?

- A function call requires space for storing variables and intermediate values



How to avoid it?

1. Use a knob to set the stack size if it easily fixes your problem …,
2. use a “balanced” recursion if possible, like:

```
def sum_range(a, b):
  if a == b:
    return 0
  else:
    c = (a + 1 + b) // 2
    return a + sum_range(a + 1, c) + sum_range(c, b)
```

3. … or use **“tail recursion”**

What is “tail recursion”?

- **Tail call** : if function f calls g, and f does nothing after g returns (other turn returns it), such a call to g is said “tail call”

```
def f(x):
  if ...:
    return g(x)      # tail call
  else:
    return g(x) + 1 # not tail call
```

- **Tail recursion** $\equiv$ recursive call that is a tail call

Tail-recursive sum_to

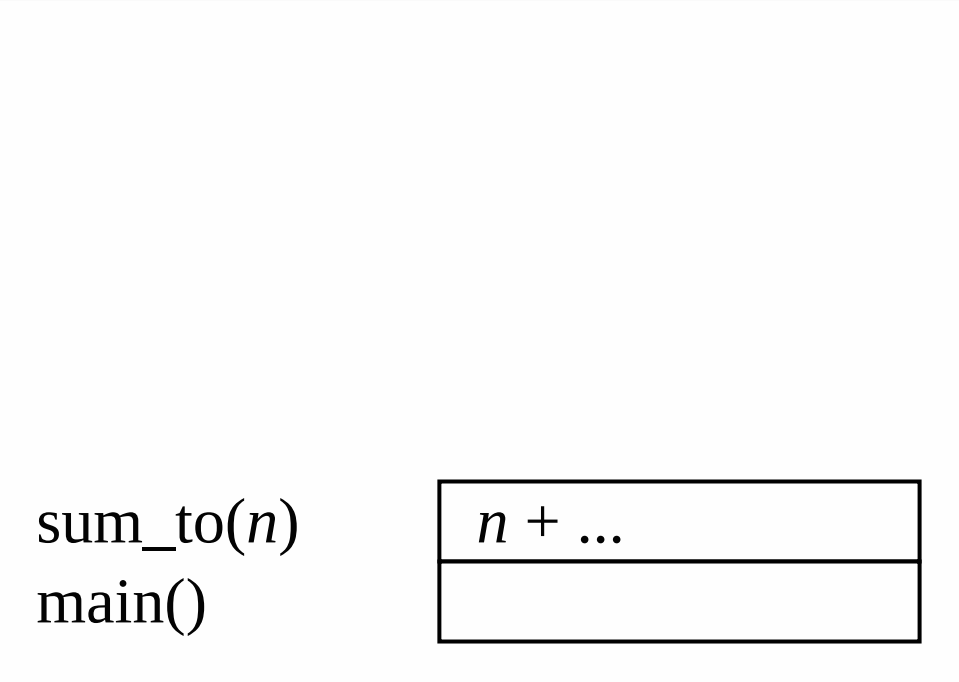
```
def sum_to_tail(n, s):
  if n == 0:
    return s
  else:
    return sum_to_tail(n - 1, n + s)

def sum_to(n):
  return sum_to_tail(n, 0)
```

- Confirm `sum_to_tail(`n`, `s`)` returns $(1 + \ldots + n) + s$

Why does it have anything to do with stack overflow?

- The true reason a function call requires space is to store values *required after the call*

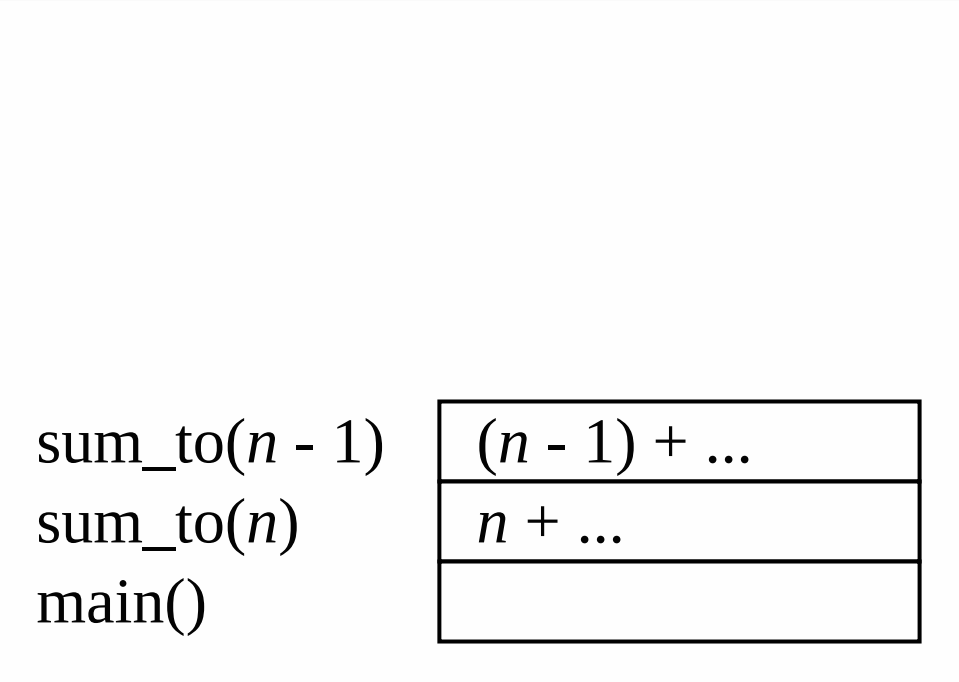
```
def sum_to(n):
  if n == 0:
    return 0
  else:
    return n + sum_to(n - 1)
```



Why does it have anything to do with stack overflow?

- The true reason a function call requires space is to store values *required after the call*

```
def sum_to(n):
  if n == 0:
    return 0
  else:
    return n + sum_to(n - 1)
```



Why does it have anything to do with stack overflow?

- The true reason a function call requires space is to store values *required after the call*

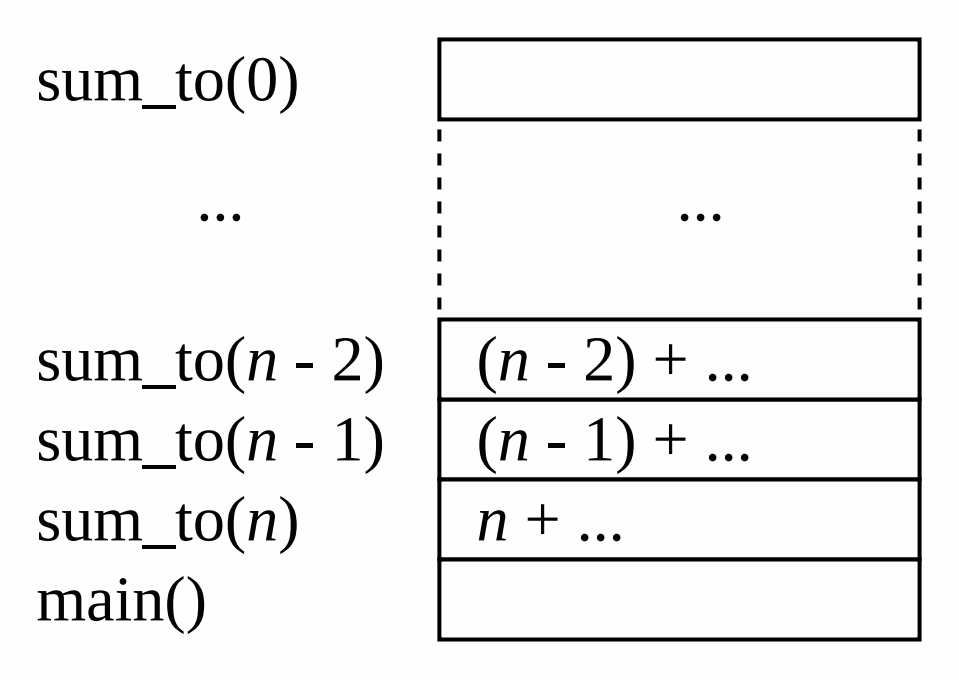
```
def sum_to(n):
  if n == 0:
    return 0
  else:
    return n + sum_to(n - 1)
```

sum_to(n - 2)	(n - 2) + ...
sum_to(n - 1)	(n - 1) + ...
sum_to(n)	n + ...
main()	

Why does it have anything to do with stack overflow?

- The true reason a function call requires space is to store values *required after the call*

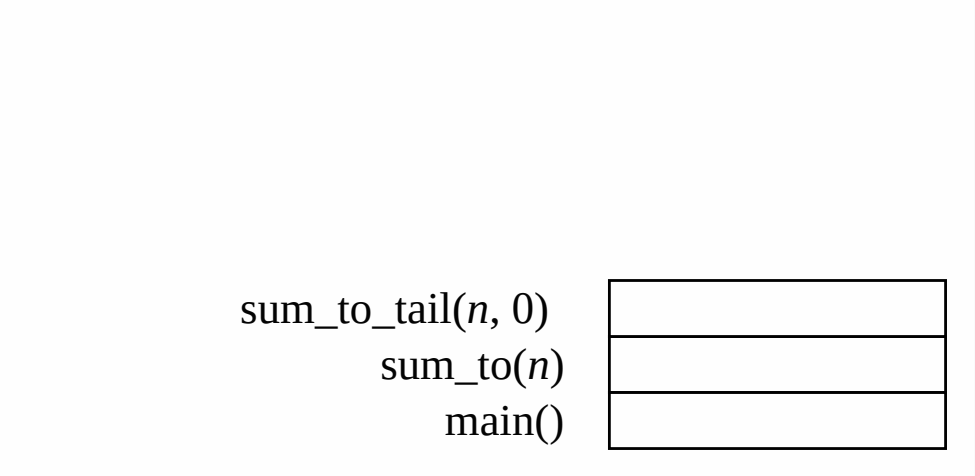
```
def sum_to(n):
  if n == 0:
    return 0
  else:
    return n + sum_to(n - 1)
```



Why does it have anything to do with stack overflow?

- But for tail calls, no space is required for computation after the call!

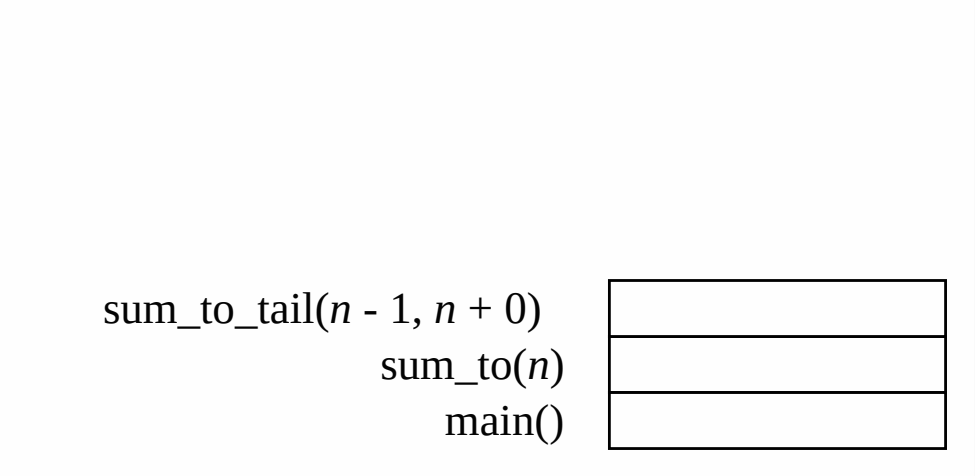
```
def sum_to_tail(n, s):
  if n == 0:
    return s
  else:
    return sum_to_tail(n - 1,
n + s)
```



Why does it have anything to do with stack overflow?

- But for tail calls, no space is required for computation after the call!

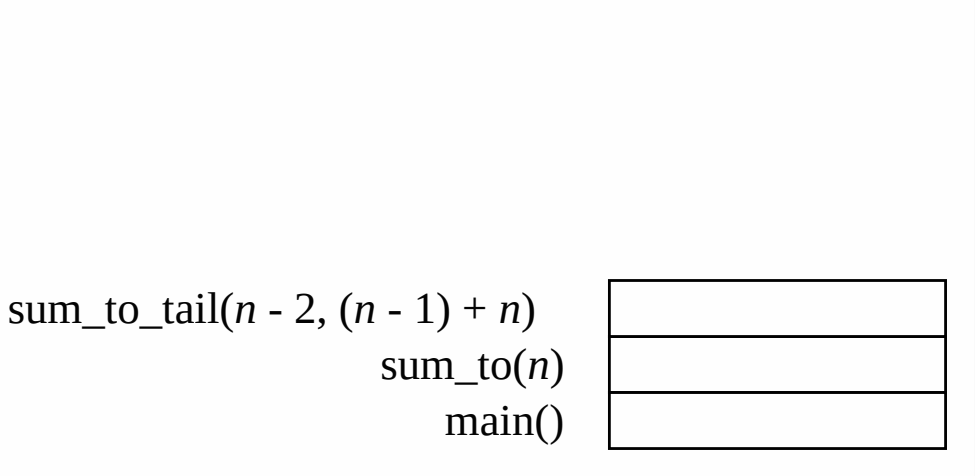
```
def sum_to_tail(n, s):
  if n == 0:
    return s
  else:
    return sum_to_tail(n - 1,
n + s)
```



Why does it have anything to do with stack overflow?

- But for tail calls, no space is required for computation after the call!

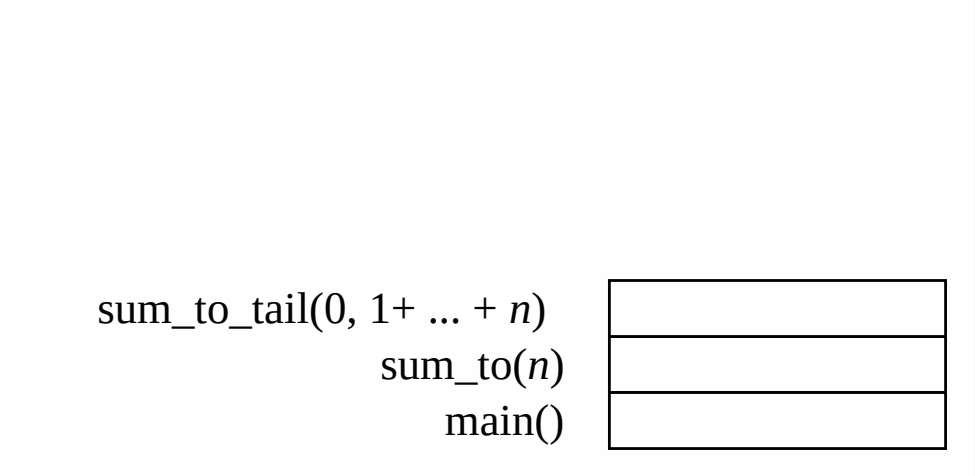
```
def sum_to_tail(n, s):
  if n == 0:
    return s
  else:
    return sum_to_tail(n - 1,
n + s)
```



Why does it have anything to do with stack overflow?

- But for tail calls, no space is required for computation after the call!

```
def sum_to_tail(n, s):
  if n == 0:
    return s
  else:
    return sum_to_tail(n - 1,
n + s)
```



How to come up with a tail-recursive version?

1. a universal formula for a simple induction like:

$$a_n = \ldots a_{n-1} \ldots$$

2. $\rightarrow$ write a function that *finds a_n given m and a_m*

```
def recurrence(m, am, n):
  if m = n:
    return am
  else:
    am_1 = ... am ... # a_{m+1}
    return recurrence(m + 1, am_1, n)
```

How to come up with a tail-recursive version?

- as the previous example indicates, taking extra parameters that represent "results of earlier/smaller cases" often does it
- another example: $a_0 = a_1 = 1; a_n = a_{n-1} + a_{n-2}$
- straight recursion:

```
def fib(n):
  if n < 2:
    return 1
  else:
    return fib(n - 1) + fib(n -2)
```

How to come up with a tail-recursive version?

- tail recursion finds a_n given a_{m-1} and a_m (note: invalid variable names used to be descriptive)

```
def fib(m, a_{m-1}, a_m, n):
  if m == n:
    return a_m
  else:
    a{m+1} = a_{m-1} + a_m
    return fib(m + 1, a_m, a_{m+1}, n)
```

Loop to tail-recursion

- following is a *general* template

```
x = x0
y = y0
while E(x, y):
  x = F(x, y)
  y = G(x, y)
return ...
```

$\Rightarrow$

```
let rec loop x y =
  if not (E x y) then
    ...
  else
    let x' = F x y in
    let y' = G x' y in
    loop x' y'
in
loop x0 y0
```

An example (sum_to)

- the natural for loop

```
s = 0
for i in range(1, n+1):
  s += i
return s
```

- while-loop version

```
i = 1
s = 0
while i <= n:
  s = s + i
  i = i + 1
return s
```

Tail recursion

```
i = 1
s = 0
while i <= n:
  s = s + i
  i = i + 1
return s
```

$\Rightarrow$

```
let rec sum_to_tail i n s =
  if i > n then
    s
  else
    sum_to_tail (i + 1) n (s + i)
in
sum_to_tail 1 n 0
```

A final remark about stack overflow

- Stack overflow is an *implementation artifact* (avoidable)
- In principle, we should be able to grow (stack + heap) up to the computer memory
- Yet, stack tends to overflow much earlier than that (e.g., a few MB, when you can use $>$ GB for other data)
- It is actually an *unnecessary* constraint, imposed by a typical/traditional memory management strategy that allocates stack separately from heap as a contiguous memory whose maximum size is set when a program (or a thread) is started

A final remark about stack overflow

- A suitable language implementation can avoid such unnecessary overflow altogether by allocating stack more flexibly (e.g., dynamically growing it by allocating stack frames from heap)
- Few language implementations (e.g., Standard ML New Jersey) do it